

Scaling Laws

- **[Kaplan]** – *Scaling laws for neural language models* (2020)
 - Language model performance follows **predictable power-law relationships** with model size, dataset size, and compute.
- **[Chinchilla]** – *Training compute-optimal large language models* (2022)
 - For a fixed compute budget, **smaller models trained on more data** achieve better performance.
- **[Beyond Chinchilla]** – *Beyond Chinchilla: Optimal- Accounting for Inference in Language Model Scaling Laws* (2023)
 - Incorporates **inference costs** into scaling laws, refining optimal model size

Why do we need scaling laws

- Imagine the following scenario..

Your friend has given you ten thousand H100s for a month, and asked you to train a good open-source LLM

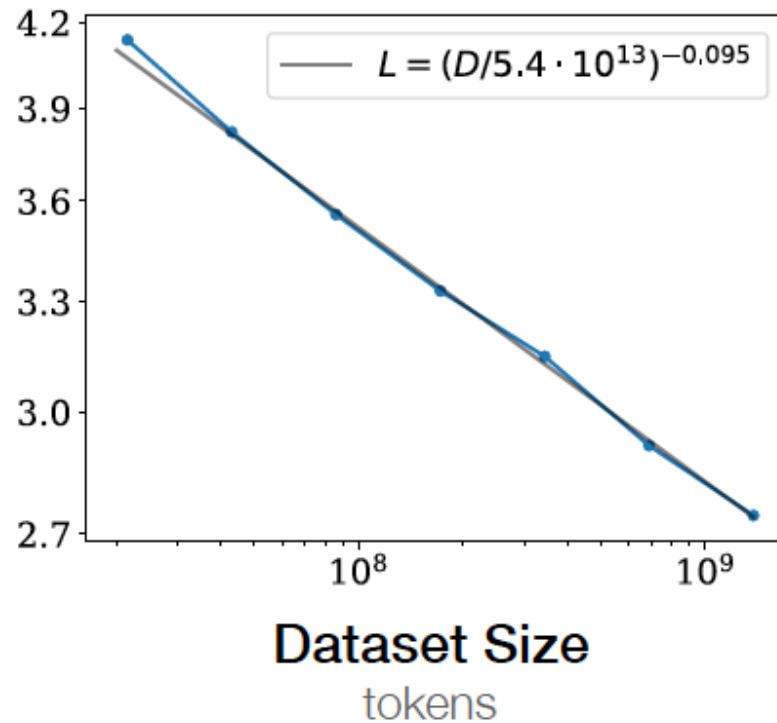
- What do you do?
 - Put together your infra team and distributed training frameworks.
 - Put together a great pretraining dataset and test suites.
 - Train the model <- we are here.

Many decisions needed

- Train a bigger model w/ less data vs. Train a small model w/ more data.
- Other hyperparameters:
 - Model shape (deeper vs. wider)
 - Learning rate
 - Batch sizes
 - Etc.
- We don't want: tune hyperparameters on big model
- Instead, tune on small models, extrapolate to large ones

Scaling laws: simple, predictive rules for model performance.

Power-law Relationship



$$L(x) = Ax^{-\alpha}$$

$$\log(L(x)) = -\alpha \log(x) + \log A$$



$$Y = -\alpha X + \log A$$

[Kaplan] – *Scaling laws for neural language models* (2020)

Settings

- **Dataset:** WebText2, contains 2.3×10^{10} tokens
- **Performance metric L :** cross-entropy loss on the test set
- **Context length:** 1,024
- **Batch size:** 512
- **Optimizer:** Adam optimizer, $1e-3$ learning rate with 3000-step linear warmup and cosine decay
- **Model size N :** ranging from 768 to 1.5B non-embedding params
- **Dataset size D :** ranging from 22 million to 23 billion tokens
- **Training compute per token:** Fwd $\sim 2N$, Bwd $\sim 4N \rightarrow 6N$
- **Batch size B , Number of training steps S**
- **Training compute $C \approx 6NBS$**
 - Measured in PF-days, $1 \text{ PF-days} = 10^{15} \times 24 \times 3600 = 8.64 \times 10^{19}$ FLOPs

1. Performance depends strongly on scale

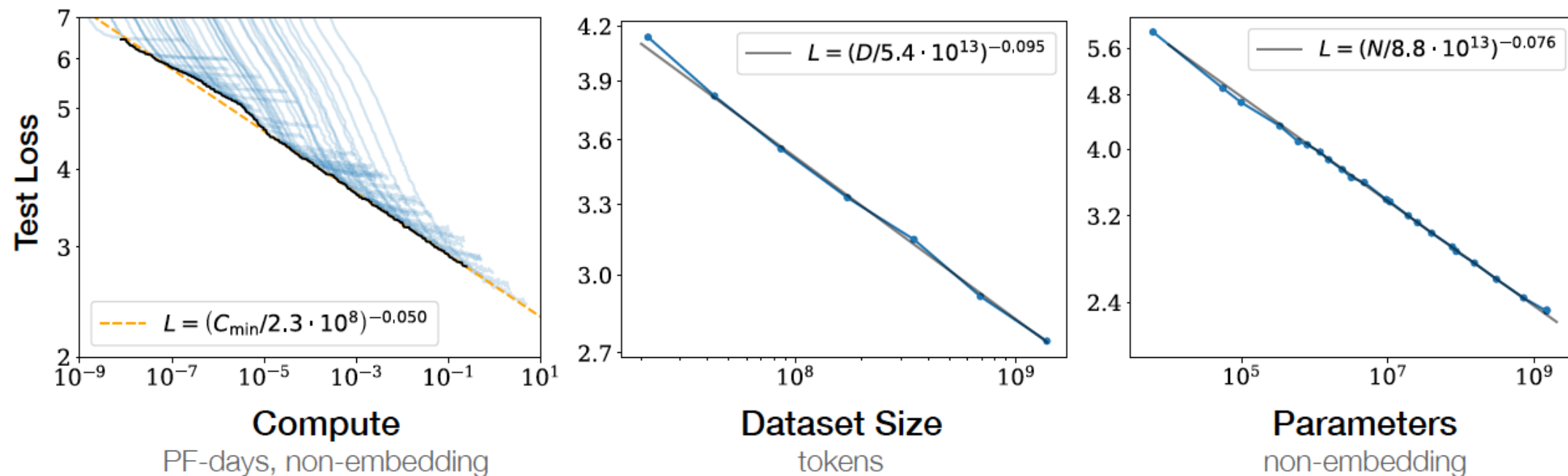
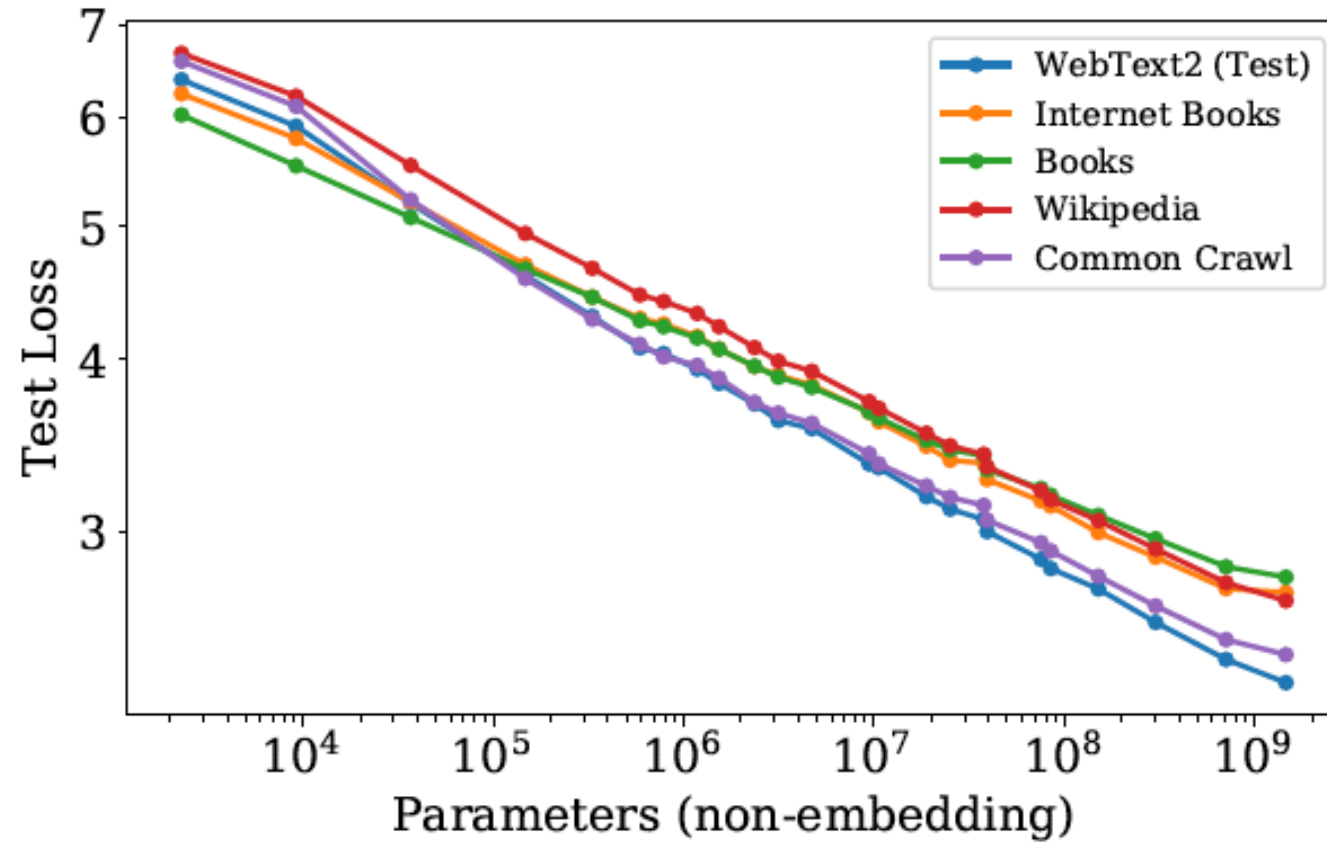


Figure 1 Language modeling performance improves smoothly as we increase the model size, dataset size, and amount of compute² used for training. For optimal performance all three factors must be scaled up in tandem. Empirical performance has a power-law relationship with each individual factor when not bottlenecked by the other two.

2. Transfer improves with test performance



3. Performance depends weakly on model shape

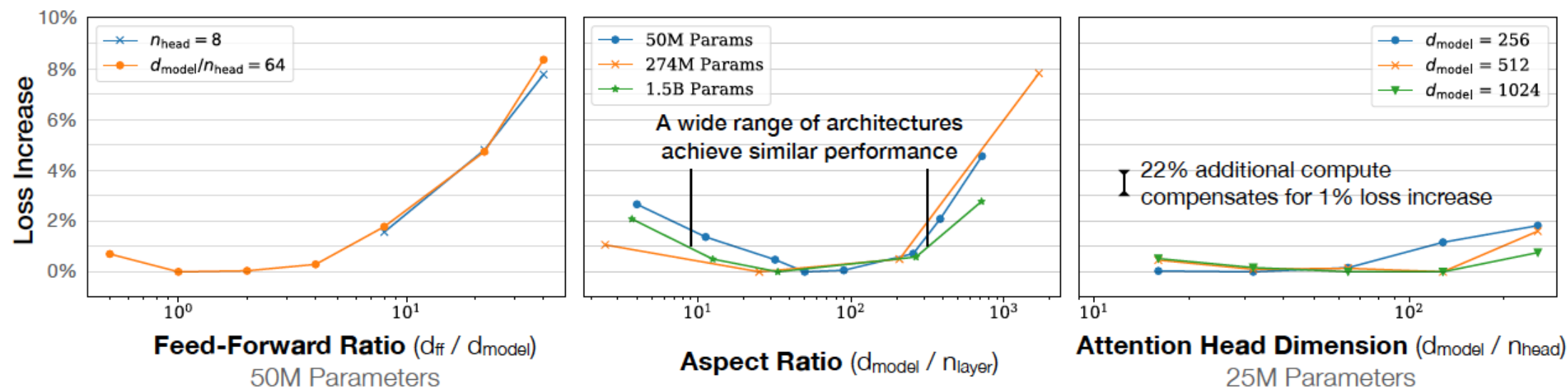
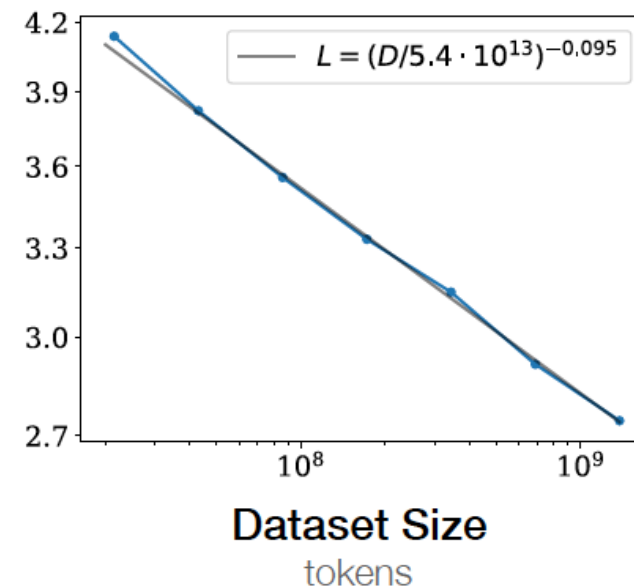
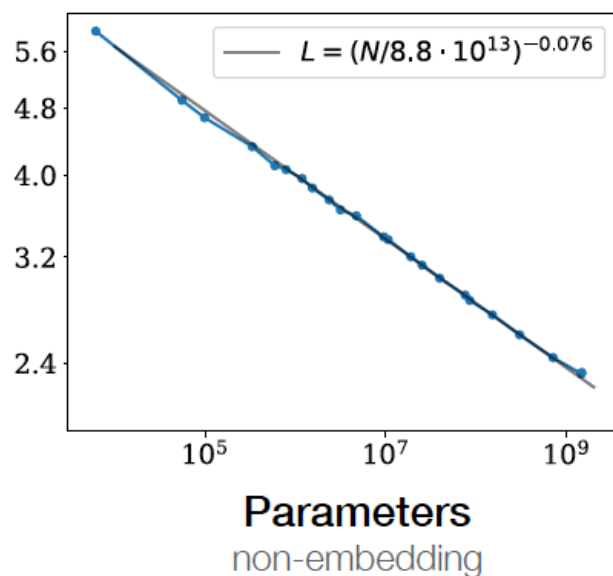


Figure 5 Performance depends very mildly on model shape when the total number of non-embedding parameters N is held fixed. The loss varies only a few percent over a wide range of shapes. Small differences in parameter counts are compensated for by using the fit to $L(N)$ as a baseline. Aspect ratio in particular can vary by a factor of 40 while only slightly impacting performance; an $(n_{layer}, d_{model}) = (6, 4288)$ reaches a loss within 3% of the $(48, 1600)$ model used in [RWC⁺19].

Usage 1: Estimate the overfit frontier



$$L(N) = (N_c/N)^{\alpha_N}; \quad \alpha_N \sim 0.076, \quad N_c \sim 8.8 \times 10^{13} \text{ (non-embedding parameters)} \quad L(D) = (D_c/D)^{\alpha_D}; \quad \alpha_D \sim 0.095, \quad D_c \sim 5.4 \times 10^{13} \text{ (tokens)}$$

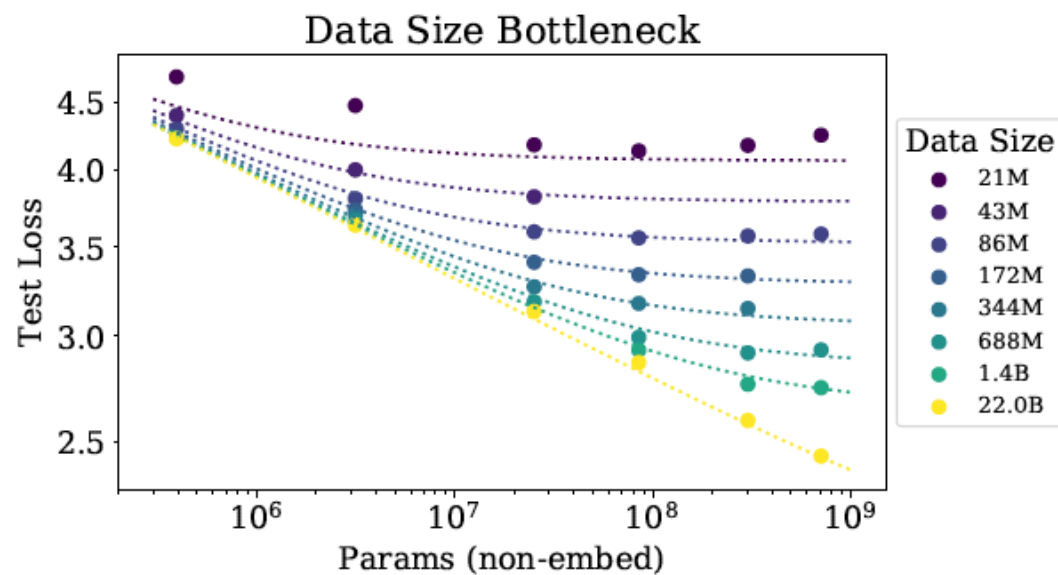


$$L(N, D) = \left[\left(\frac{N_c}{N} \right)^{\frac{\alpha_N}{\alpha_D}} + \frac{D_c}{D} \right]^{\alpha_D}$$

Parameter	α_N	α_D	N_c	D_c
Value	0.076	0.103	6.4×10^{13}	1.8×10^{13}

Table 2 Fits to $L(N, D)$

Usage 1: Estimate the overfit frontier



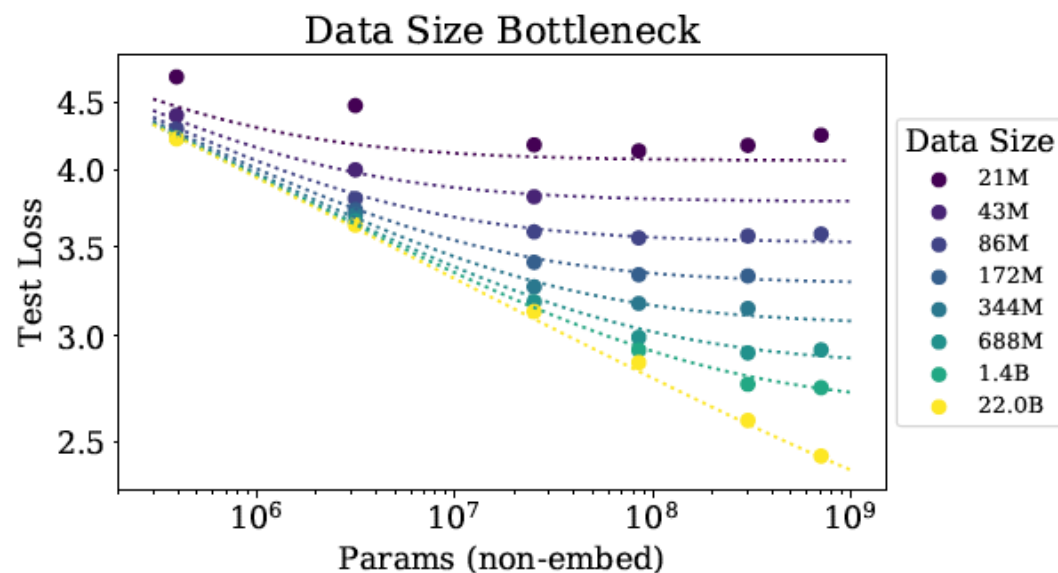
➡

$$L(N, D) = \left[\left(\frac{N_c}{N} \right)^{\frac{\alpha_N}{\alpha_D}} + \frac{D_c}{D} \right]^{\alpha_D}$$

Parameter	α_N	α_D	N_c	D_c
Value	0.076	0.103	6.4×10^{13}	1.8×10^{13}

Table 2 Fits to $L(N, D)$

Usage 1: Estimate the overfit frontier



$$\delta L(N, D) \equiv \frac{L(N, D)}{L(N, \infty)} - 1$$

$$\delta L \approx \left(1 + \left(\frac{N}{N_c} \right)^{\frac{\alpha_N}{\alpha_D}} \frac{D_c}{D} \right)^{\alpha_D} - 1$$

$$\frac{\alpha_N}{\alpha_D} \approx 0.74 \quad D \gtrsim (5 \times 10^3) N^{0.74}$$

Dataset size may grow sub-linearly in model size while avoiding overfitting. Every time we increase the model size 8x, we need to increase the data by roughly 5x.

➡

$$L(N, D) = \left[\left(\frac{N_c}{N} \right)^{\frac{\alpha_N}{\alpha_D}} + \frac{D_c}{D} \right]^{\alpha_D}$$

Parameter	α_N	α_D	N_c	D_c
Value	0.076	0.103	6.4×10^{13}	1.8×10^{13}

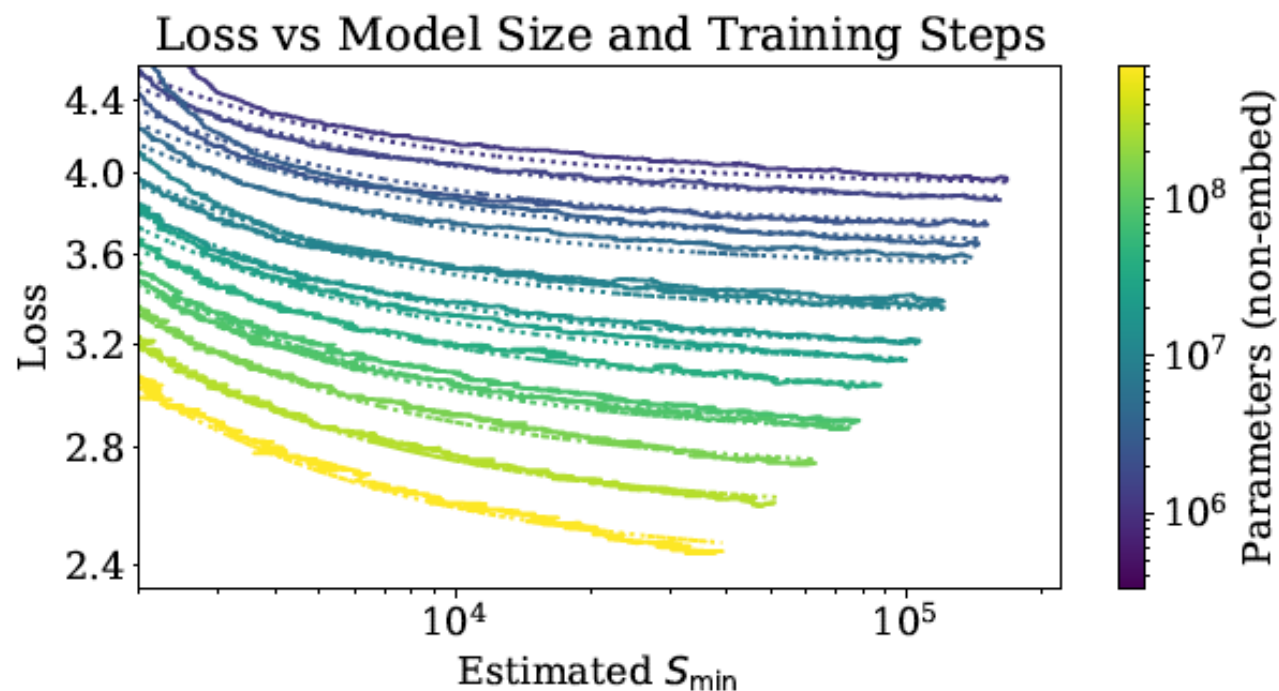
Table 2 Fits to $L(N, D)$

Usage 2: Estimate optimal allocation of the compute

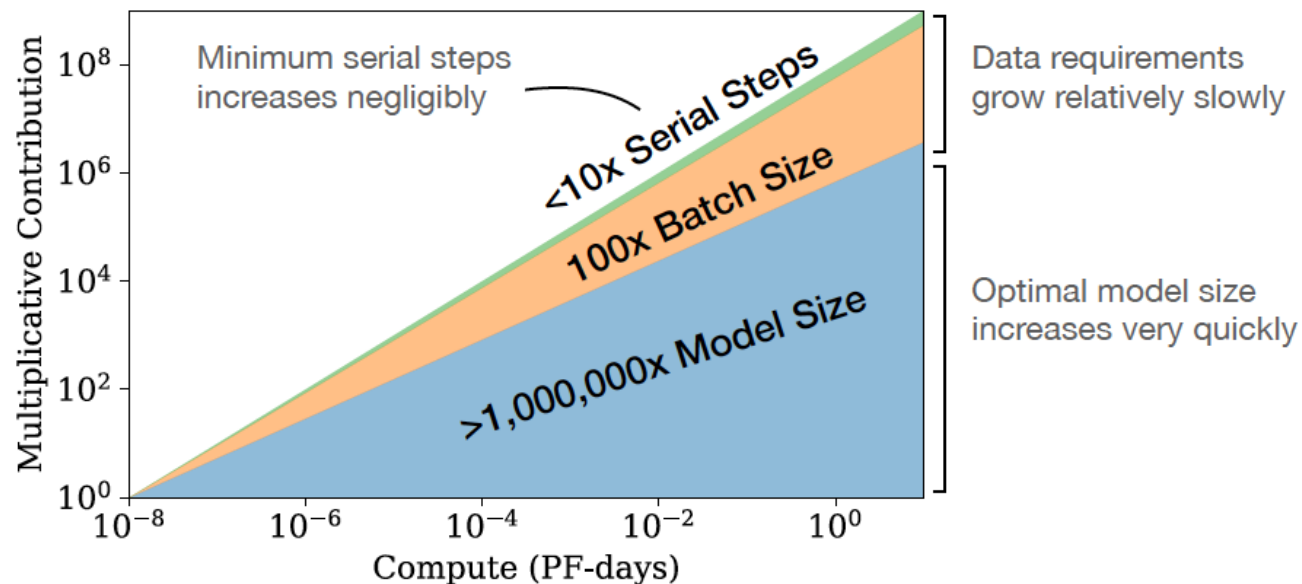
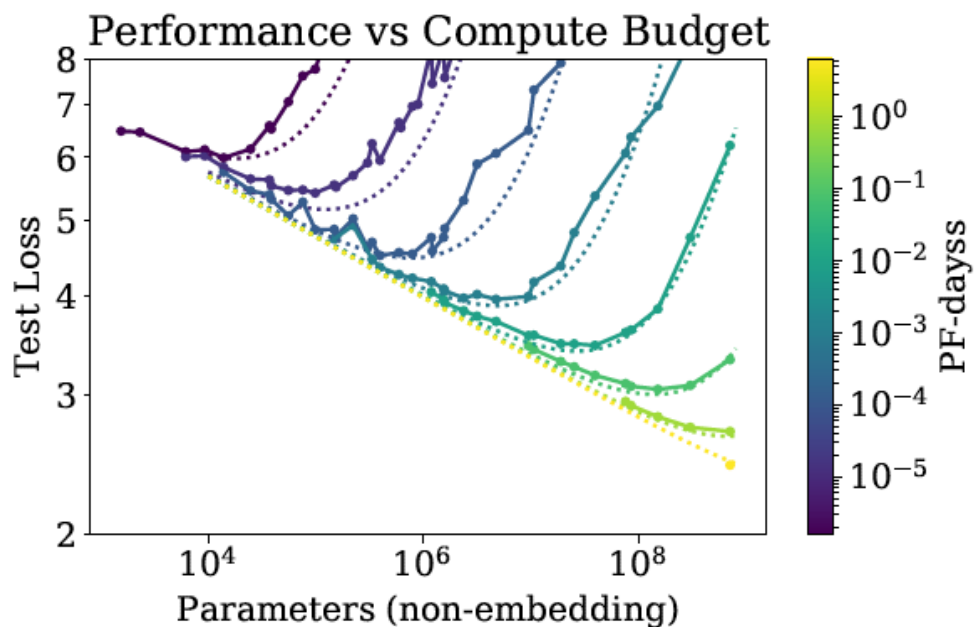
$$L(N, S_{\min}) = \left(\frac{N_c}{N}\right)^{\alpha_N} + \left(\frac{S_c}{S_{\min}}\right)^{\alpha_S}$$

Parameter	α_N	α_S	N_c	S_c
Value	0.077	0.76	6.5×10^{13}	2.1×10^3

Table 3 Fits to $L(N, S)$



Usage 2: Estimate optimal allocation of the compute

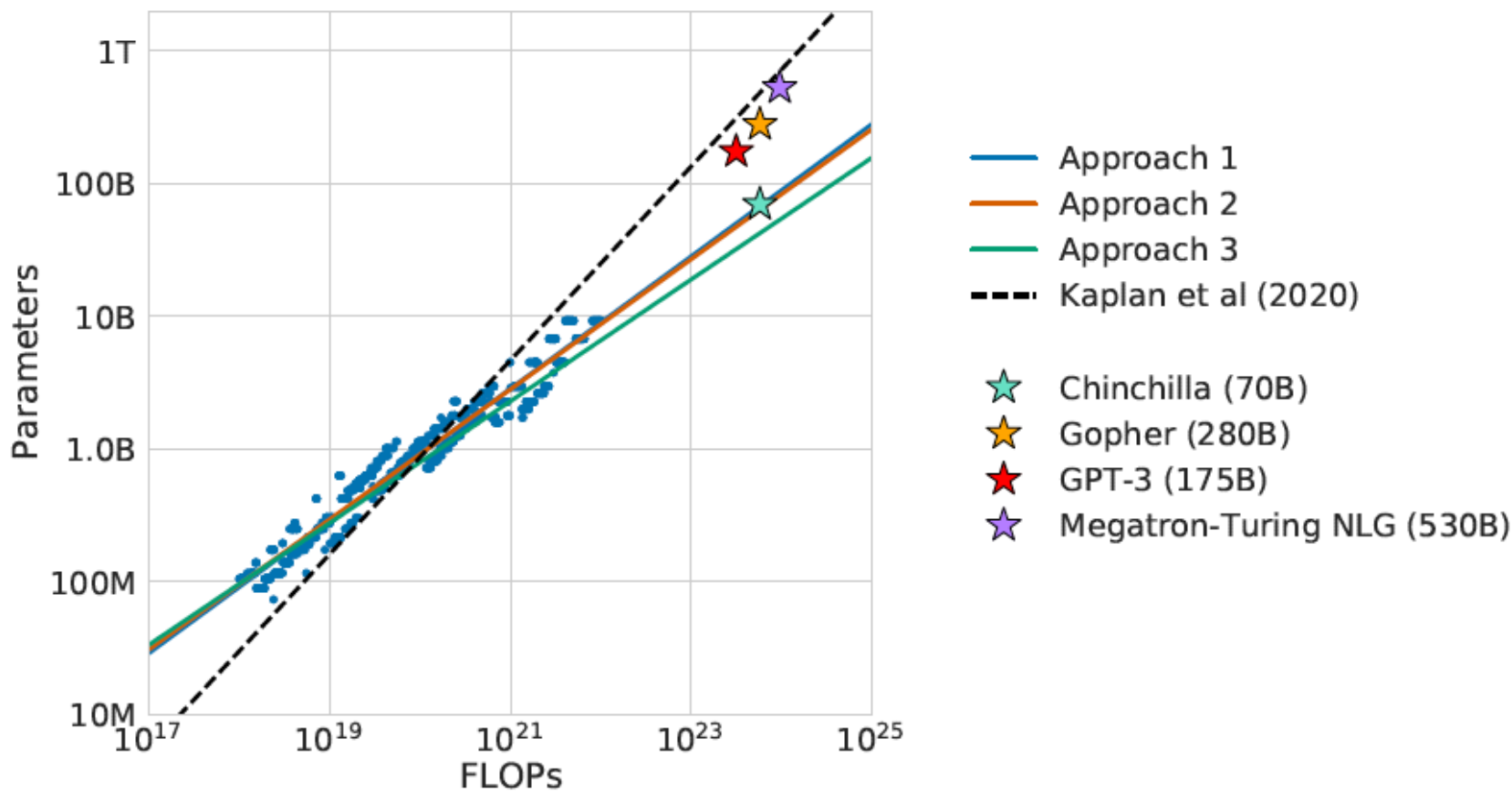


Takeaways

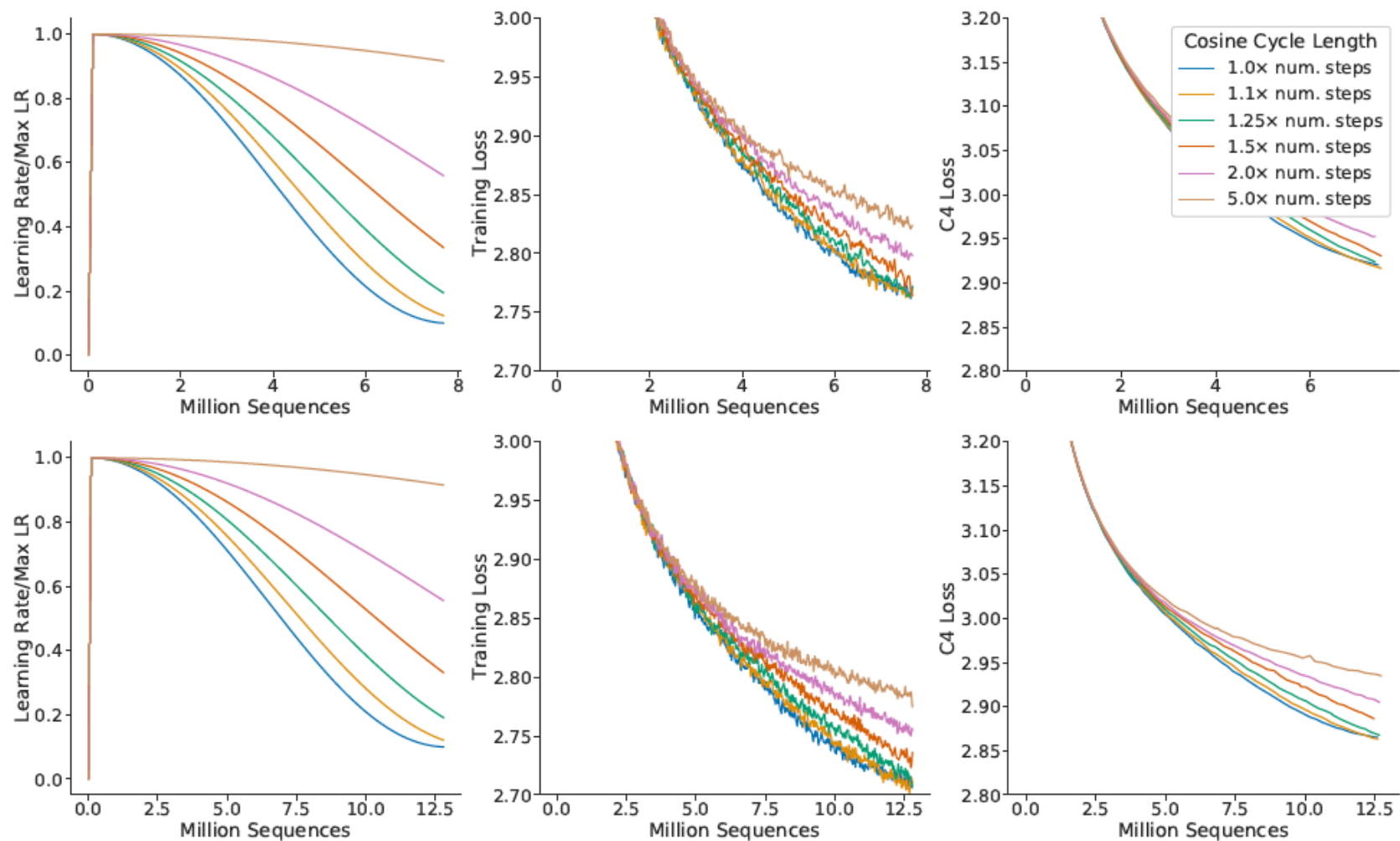
- Performance has a power-law relationship with N, D, C when not bottlenecked by the other two. It depends very weakly on other architectural hyperparameters.
- Performance will have diminishing returns if either N or D is fixed while the other increases. To avoid overfitting, D may grow sub-linearly in N ($D \sim N^{0.74}$).
- When working within a fixed compute budget, we attain optimal performance by training very large models on a relatively modest amount of data.

[Chinchilla] – *Training compute-optimal large language models* (2022)

Previous work over-emphasized parameter growth



Problem: Learning rate schedule



Chinchilla in depth – 3 methods

Approach	Coeff. a where $N_{opt} \propto C^a$	Coeff. b where $D_{opt} \propto C^b$
1. Minimum over training curves	0.50 (0.488, 0.502)	0.50 (0.501, 0.512)
2. IsoFLOP profiles	0.49 (0.462, 0.534)	0.51 (0.483, 0.529)
3. Parametric modelling of the loss	0.46 (0.454, 0.455)	0.54 (0.542, 0.543)
Kaplan et al. (2020)	0.73	0.27

Method 1 – minimum over runs.

Similar to the FLOPS figure on Kaplan – the minimum over the union of all training curves is a power law.

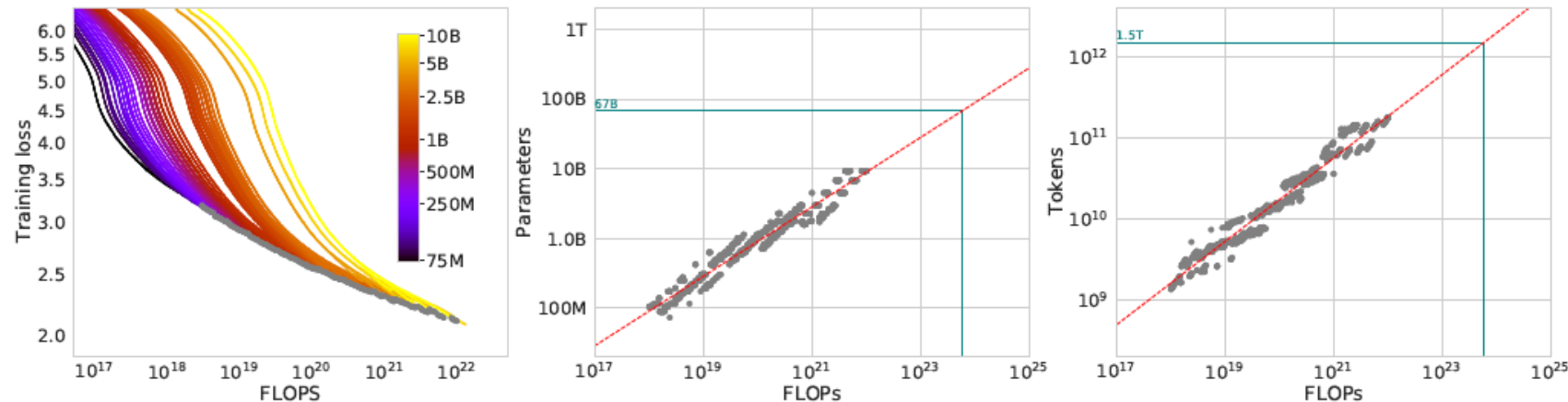


Figure 2 | **Training curve envelope.** On the **left** we show all of our different runs. We launched a range of model sizes going from 70M to 10B, each for four different cosine cycle lengths. From these curves, we extracted the envelope of minimal loss per FLOP, and we used these points to estimate the optimal model size (**center**) for a given compute budget and the optimal number of training tokens (**right**). In green, we show projections of optimal model size and training token count based on the number of FLOPs used to train *Gopher* (5.76×10^{23}).

Method 2 - IsoFLOPS

Pick a range of FLOP budgets, vary the total parameter count, take the min over these convex shapes. The minima form a power law.

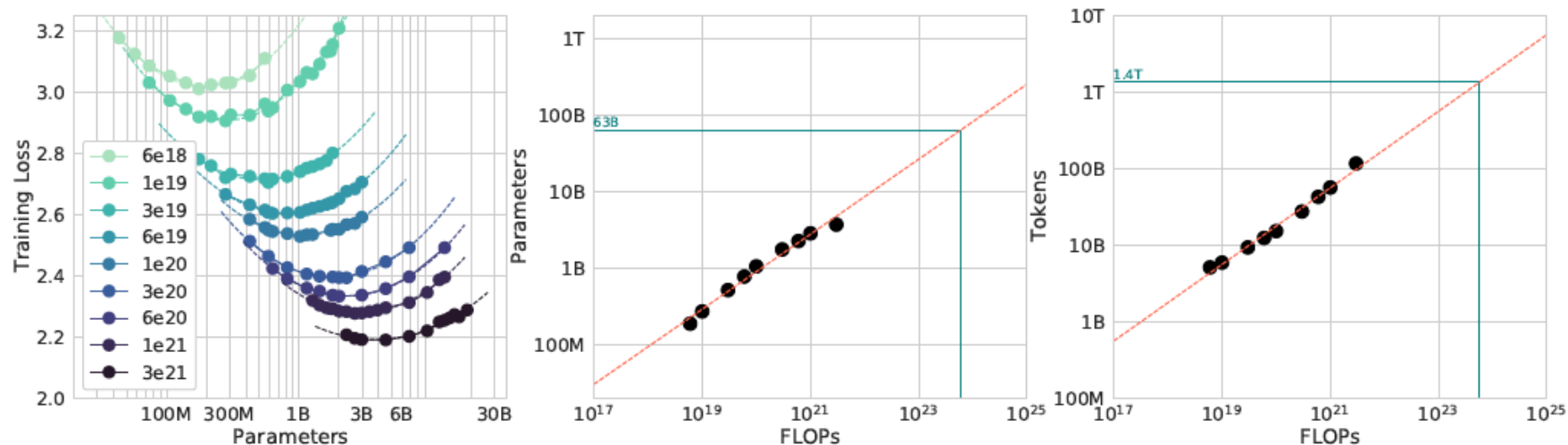


Figure 3 | **IsoFLOP curves.** For various model sizes, we choose the number of training tokens such that the final FLOPs is a constant. The cosine cycle length is set to match the target FLOP count. We find a clear valley in loss, meaning that for a given FLOP budget there is an optimal model to train (**left**). Using the location of these valleys, we project optimal model size and number of tokens for larger models (**center** and **right**). In green, we show the estimated number of parameters and tokens for an *optimal* model trained with the compute budget of *Gopher*.

Method 3 – Joint fits

Run a bunch of models on the size-data grid. Use least squares to fit a joint scaling law

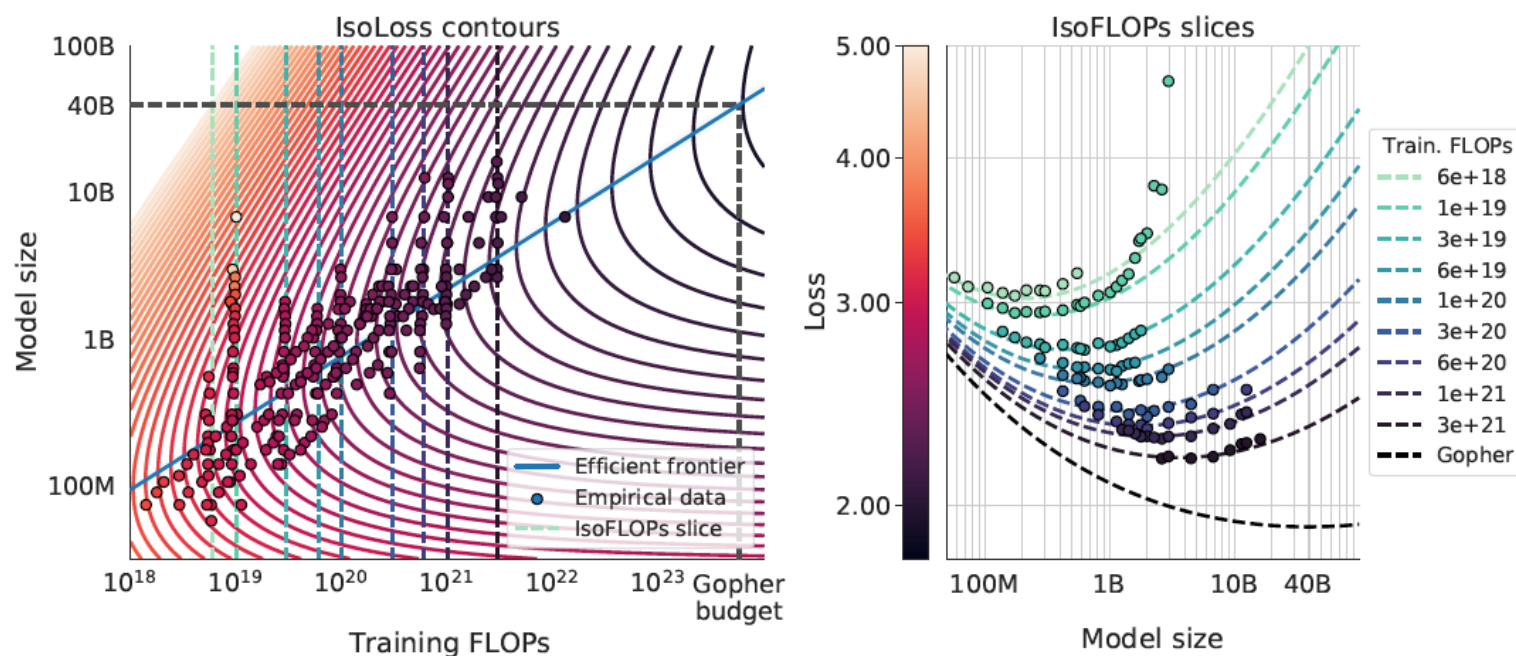


Figure 4 | **Parametric fit.** We fit a parametric modelling of the loss $\hat{L}(N, D)$ and display contour (**left**) and isoFLOP slices (**right**). For each isoFLOP slice, we include a corresponding dashed line in the left plot. In the left plot, we show the efficient frontier in blue, which is a line in log-log space. Specifically, the curve goes through each iso-loss contour at the point with the fewest FLOPs. We project the optimal model size given the *Gopher* FLOP budget to be 40B parameters.

Chinchilla– ‘overtrain’ for compute-optimal

Approach	Coeff. a where $N_{opt} \propto C^a$	Coeff. b where $D_{opt} \propto C^b$
1. Minimum over training curves	0.50 (0.488, 0.502)	0.50 (0.501, 0.512)
2. IsoFLOP profiles	0.49 (0.462, 0.534)	0.51 (0.483, 0.529)
3. Parametric modelling of the loss	0.46 (0.454, 0.455)	0.54 (0.542, 0.543)
Kaplan et al. (2020)	0.73	0.27

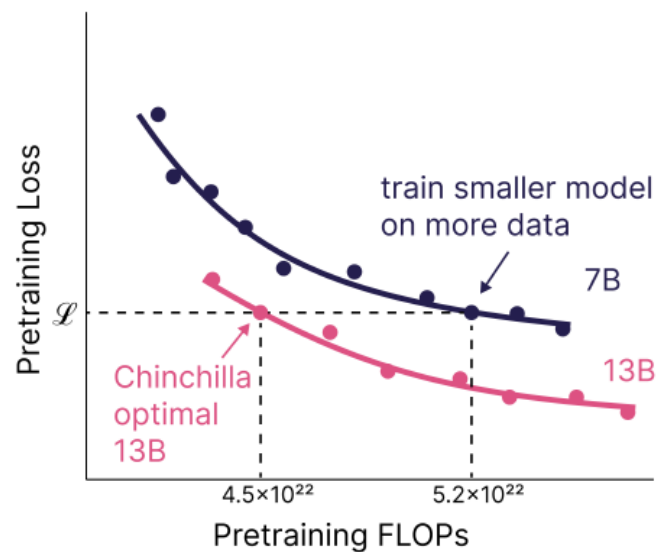
- **GPT3** – 2 tokens / param
- **Chinchilla** – 20 tokens / param
- **LLaMA65B** – 22 tokens / param
- **Llama 2 70B** – 29 tokens / param
- **Mistral 7B** – 110 tokens / param
- **Llama 3 70B** – 215 tokens / param

Train Chinchilla, a 70B model on 1.4 trillion tokens. Using the same compute budget, Chinchilla beats Gopher (280B, ~300B tokens) broadly; being smaller and cheaper for inference and fine-tuning.

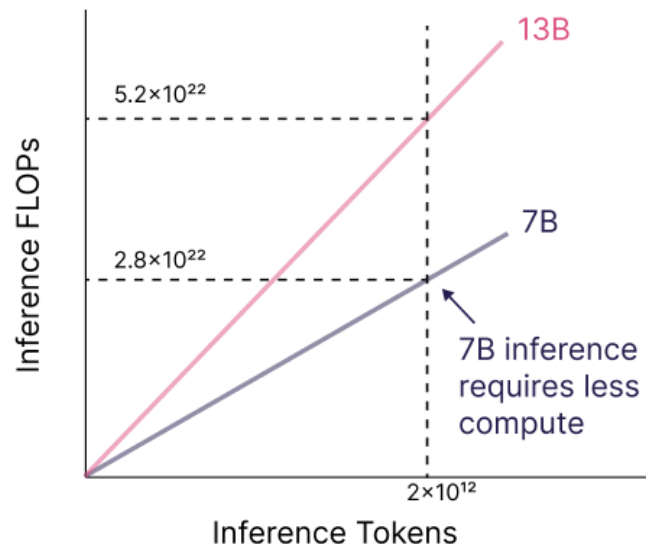
[Beyond Chinchilla] – *Beyond Chinchilla: Optimal-Accounting for Inference in Language Model Scaling Laws (2023)*

Accounting for inference in scaling laws

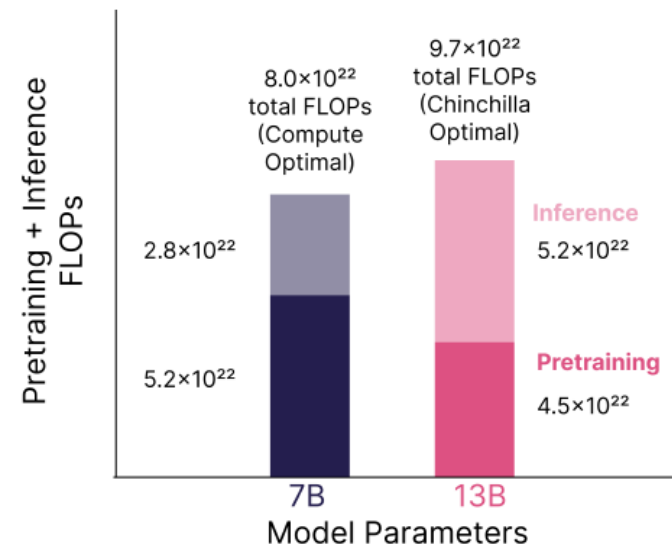
$$N^*(\ell, D_{\text{inf}}), D_{\text{tr}}^*(\ell, D_{\text{inf}}) = \arg \min_{N, D_{\text{tr}} | L(N, D_{\text{tr}}) = \ell} 6ND_{\text{tr}} + 2ND_{\text{inf}}. \quad (3)$$



(a)



(b)



(c)

Accounting for inference in scaling laws

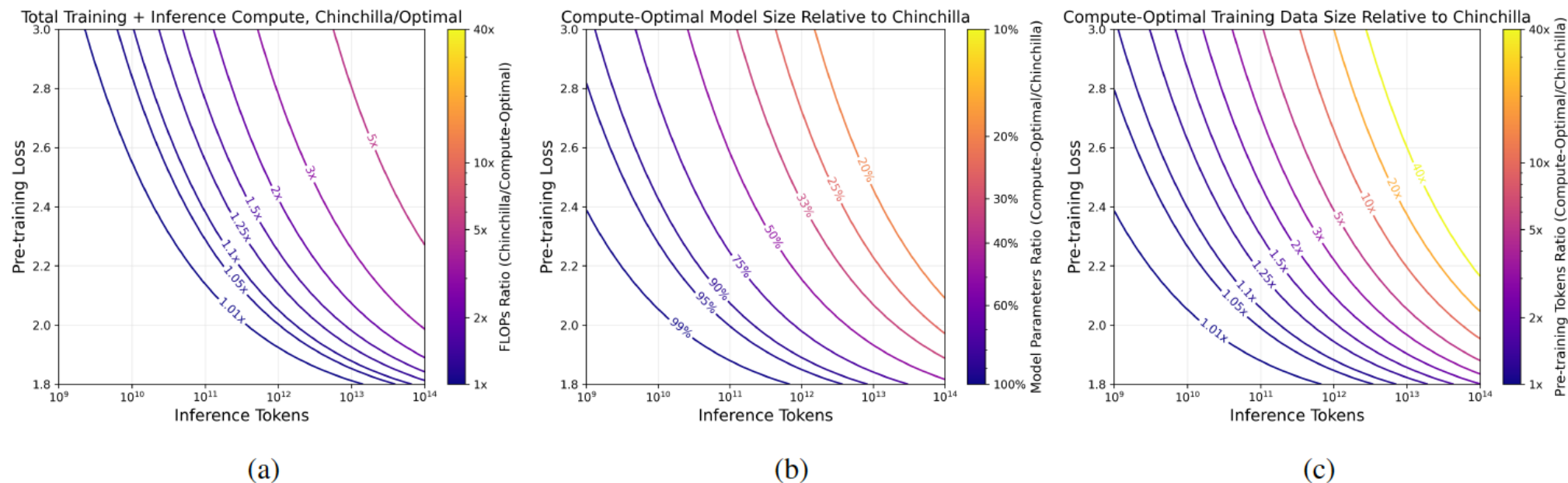


Figure 2: Ratios of (a) total FLOPs, (b) model parameters, and (c) pre-training tokens, for optimal models estimated via our method vs. Chinchilla-style models. For each point (x, y) in the figures, we compute the Chinchilla model parameter count and training data required to reach the loss y , and the number of combined FLOPs required to train and run inference for x tokens using the Chinchilla model. Then, we compute the same values (total FLOPs, parameter count, training data size) for the compute-optimal models returned by our method, and plot the ratios.

- **[Kaplan]** – *Scaling laws for neural language models* (2020)
 - Language model performance follows **predictable power-law relationships** with model size, dataset size, and compute.
- **[Chinchilla]** – *Training compute-optimal large language models* (2022)
 - For a fixed compute budget, **smaller models trained on more data** achieve better performance.
- **[Beyond Chinchilla]** – *Beyond Chinchilla: Optimal- Accounting for Inference in Language Model Scaling Laws* (2023)
 - Incorporates **inference costs** into scaling laws, refining optimal model size